# CS6008 Network Security, Assignment 2: DNS Security

Report

Shiv Narang (23B1075) and Sabil Ahmed (23B1057)

October 2026

## Overview

We build a small DNS hierarchy in Docker across three isolated subnets that are joined only by two no-NAT routers. First we run plain DNS, then attack it from an on-path position, then add defences one at a time (DNSSEC, and finally an encrypted last mile) and see which attack each one actually stops.

**Addresses.** client `172.16.1.10`, webserver `172.16.1.20`, rogue-web `172.16.1.30`; resolver `172.16.2.10`, attacker `172.16.2.100` (phases 2 to 5) or `172.16.1.100` (phases 6 to 7); root-ns `172.16.3.10`, tld-ns `172.16.3.20`, domain-ns `172.16.3.30`; edge-router `172.16.1.2`/`172.16.2.2`, core-router `172.16.2.3`/`172.16.3.3`.

**Shared code (`common/`).** `base/` has the Phase 1 hierarchy and the BIND 9.20 image, `signing/` does the DNSSEC signing, and `interceptor/` is the C/libpcap attack tool. Every `phaseN/` folder is self-contained and reuses these.

We ran everything on Colima (Docker 29, Ubuntu 24.04 VM, Apple Silicon). Each result below comes from the `scripts/verify.sh` or `scripts/attack.sh` in that phase's folder.

## Phase 1: Building the DNS Hierarchy

### Design choices

Isolation comes from the topology, not from firewall rules. Each subnet is a Docker bridge with `internal: true`, so there is no NAT and no Internet. A packet only crosses subnets through a router that sits on both bridges, and the forwarding happens inside that router container, so we never have to fight Docker's own bridge isolation. Nothing else forwards.

Each node installs its own static routes from `STATIC_ROUTES` in a small entrypoint, which only needs `NET_ADMIN`: client reaches the resolver net via edge-router, the resolver reaches the auth net via core-router and the client net via edge-router, and each nameserver reaches the resolver net via core-router. The routers need no extra routes since both of their subnets are directly connected.

Addresses stay real end to end: root hints and glue point at the nameservers' `172.16.3.x` addresses. The resolver is recursive with DNSSEC validation off.

### Report answers

Resolution, fetch, and the isolation check:

```
$ docker exec client dig +short example.com
172.16.1.20
$ docker exec client dig +short www.example.com
172.16.1.20
$ docker exec client curl -s http://example.com | grep -i welcome
  <h1>Welcome to example.com</h1>
$ docker exec client dig +time=2 +tries=1 @172.16.3.10 example.com
```

```
;; no servers could be reached        # auth subnet is not directly reachable
```

The referral chain, from a capture on core-router. One `example.com/A` resolution crosses it as these exchanges (counted by destination):

```
16 queries -> 172.16.3.10 (root-ns)    # . and com, plus resolving the NS names
 2 queries -> 172.16.3.20 (tld-ns)     # referral to example.com + glue
 2 queries -> 172.16.3.30 (domain-ns)  # final A answer 172.16.1.20
```

Everything on the wire is cleartext. You can read the QNAME being asked, the 16-bit query ID and UDP source port that tie a reply to its query, the record type, and the answers (the referral NS names with their glue, then `example.com A 172.16.1.20`). The root is hit many times only because our nameserver names live in their own pseudo-TLDs; resolution still finishes from the glue.

How the resolver's query reaches the root:

```
$ docker exec resolver ip route get 172.16.3.10
172.16.3.10 via 172.16.2.3 dev eth0 src 172.16.2.10
```

The resolver has no leg on the auth subnet, so it uses the static route `172.16.3.0/24 via 172.16.2.3` and hands the packet to core-router, which forwards it out of its `172.16.3.3` interface to root-ns.

Because the whole exchange is unauthenticated cleartext, any node on the path can read a query and forge a reply by copying its QNAME, ID, and ports. That is exactly Phase 2.

# Phase 2: On-Path Response Forgery

## Design choices

The diversion is a single route on the router. We add `ip route replace 172.16.2.10/32 via 172.16.2.100` on core-router, so the resolver's inbound responses go to the attacker first. The resolver and nameservers keep their Phase 1 config.

The attacker does not forward. It has `NET_RAW` (plus `NET_ADMIN`) and runs the C tool in `common/interceptor/dns_intercept.c` Since the kernel would drop the diverted frames (wrong destination IP, and `ip_forward=0`), we use `pcap_open_live` to capture and `pcap_inject` to relay them, which is the capture-and-inject path the brief asks for.

The forge is selective. A BPF filter picks only the diverted DNS responses (`ether dst <us> and udp and src port 53 and dst host 172.16.2.10`), and never our own injected frames, so there is no loop. We rewrite the A record only when the question is exactly `example.com` with type A. IP and UDP checksums are recomputed and the packet length does not change.

## Evidence

```
# relay mode (forging off): transparent
example.com     -> 172.16.1.20   ("Welcome to example.com")

# forge mode:
example.com     -> 172.16.1.30   (rogue "HACKED" page)
www.example.com -> 172.16.1.20   (left alone)
```

## Report answers

Plain DNS has no origin authentication. A resolver accepts any reply that matches the query's QNAME, type, 16-bit ID, and UDP port. An off-path spoofer has to guess the ID and port and win a race against the real server. Our on-path attacker just reads them from the genuine packet and rewrites it, so there is nothing to guess and no race, and the attack is deterministic.

Non-target names are safe because the tool only rewrites answers when the QNAME is exactly `example.com` and the type is A. `www.example.com` is a different QNAME, so it falls through the relay path and is injected byte for byte, which is why it still returns `172.16.1.20`.

# Phase 3: DNSSEC Chain of Trust

## Design choices

We sign with ECDSAP256SHA256 (algorithm 13). The keys and signatures are small and validation is fast, and it is the modern default. The trade-off is a shorter track record than RSA and slightly less universal support than it is new (Ed25519 is even newer), but it keeps the signed zones compact and is widely supported.

The chain is built child first (`common/signing/sign-all.sh`): sign `example.com`, put its KSK's DS into `com`, sign `com`, put its DS into the root, then sign the root. The resolver anchors on the root KSK only (`trust-anchor.conf`) and turns on `dnssec-validation yes`. Root hints are unchanged from Phase 1. The attacker and its Phase 2 binary are untouched.

No private key is in the repository. Keys and signed zones are build artifacts that the signer regenerates, and they are git-ignored.

## Verify (healthy signed hierarchy)

```
$ dig +dnssec example.com   ->  status: NOERROR; flags: qr rd ra ad ; + RRSIG
$ delv -a trust-anchor.conf @172.16.2.10 example.com
; fully validated
example.com.  3582  IN  A  172.16.1.20
```

## Attack (Phase 2 forgery re-run, unchanged)

```
$ docker exec client dig example.com
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL        # forged answer rejected
$ docker exec client dig +short www.example.com
172.16.1.20                                           # untargeted name still validates
# resolver log:
validating example.com/A: no valid signature found
RRSIG failed to verify resolving 'example.com/A/IN': 172.16.3.30#53
# client fetch: no page, resolution failed, attack defeated
```

## Report answers

The attacker injects the same forged A, but it cannot produce a matching RRSIG because it has no zone private key. The validating resolver walks the chain (root KSK, DS(com), com KSK, DS(example.com), example.com ZSK) and checks the signature over the A RRset. It finds no valid signature, so it cannot call the name secure and returns SERVFAIL instead of the poison. In Phase 2 the same packet was accepted because nothing was checked.

On the wire the exchange is still cleartext, so the on-path attacker can read the QNAME, the IDs, and the answers just like in Phase 1, now with the RRSIG, DNSKEY, and DS records visible too. So DNSSEC gives origin authentication and integrity of the DNS data, but not confidentiality, and it does not protect the last hop to a stub that does not validate. That last hop is Phase 6.

Private keys do not belong in the repo. We commit only the signing scripts and the public trust anchor; the KSK and ZSK private keys are regenerated each run.

# Phase 4: Signature Stripping and Downgrade

## Design choices

Forging a signed answer failed in Phase 3, so here the attacker tries to make the signed zone look unsigned and get the plain Phase 2 forgery accepted again. The interceptor gets a third mode, `strip` (see `handle_dns()`): for `example.com` it rewrites the A to the rogue and cuts the message off right after that record, so the RRSIG and every other DNSSEC record is gone and the resolver is handed a bare, unsigned-looking A. The attacker holds no private keys.

The two runs differ by one line in the signer (`NODS` in `sign-all.sh`), which tells `com` whether to publish the DS for `example.com`.

## Run A: com publishes the DS (secure delegation)

```
# baseline (no attacker): example.com validates
;; flags: qr rd ra ad                                  # authenticated
# strip attack:
$ docker exec client dig example.com
;; ->>HEADER<<- status: SERVFAIL
# resolver log:
insecurity proof failed resolving 'example.com/A/IN': 172.16.3.30#53
```

## Run B: com publishes no DS (insecure delegation)

```
# baseline (no attacker): example.com resolves but is NOT authenticated
;; flags: qr rd ra                                     # no ad flag, insecure
# same strip attack now lands:
$ docker exec client dig +short example.com
172.16.1.30                                           # rogue "HACKED" page
```

## Report answers

In Run A the signatures, and even the DS, can be stripped on the wire, but the attacker still cannot make the zone look unsigned. "Unsigned" is not the absence of signatures, it is a signed statement by the parent that the child has no DS. `com` still says, with a signature the attacker cannot forge, that `example.com` is signed, so when the resolver is handed an unsigned answer for a name that must be signed it returns SERVFAIL ("insecurity proof failed") rather than trust it.

In Run B the single change is dropping `example.com`'s DS from `com`. That flips the parent's signed statement to "this child is insecure", `com` now carries a signed NSEC proving no DS exists, and the resolver believes it because it is authenticated. From then on `example.com` is plain DNS and the Phase 2 forgery works again.

So downgrade resistance does not come from the child being signed. It comes from the secure delegation: the DS in the parent, chained to the trust anchor. The exact operator mistake that forfeits it is signing a zone but never publishing (or later dropping) its DS in the parent.

# Phase 5: Zone Walking

## Design choices

A signed zone must prove a name does not exist, and that proof is signed ahead of time, before anyone knows which name will be asked. It cannot be about the queried name, so it reveals something about the names that do exist. With NSEC, what it reveals is the next existing name. Our `example.com` advertises only `example.com` and `www`; the rest (`backup, dev, git, mail, staging, vpn`) are internal names that appear nowhere public.

### Verify and attack

A nonexistent name is denied with signed NSEC records that each point from one real name to the next, which leaks a real neighbour. Following that chain from the apex recovers the whole zone, one query per name (`scripts/zonewalk.sh`, which uses `dig`, no Scapy or dnspython):

```
$ sh scripts/walk.sh
queries sent : 9
names recovered:
  backup.example.com. 10.0.0.5     dev.example.com.  10.0.0.6
  git.example.com.    10.0.0.7     mail.example.com. 10.0.0.8
  ns.example.com.     172.16.3.30  staging...  10.0.0.9   vpn... 10.0.0.10
  www.example.com.    172.16.1.20
```

Nine names in nine queries, inside the (names + 2) budget.

#### Fix

Re-signing with NSEC3 (`NSEC3=1 sh sign.sh`) hashes the owner names before chaining them. The same walk now returns no NSEC to follow and recovers nothing, while a nonexistent name still gets an authenticated denial, now via NSEC3.

### Report answers

NSEC made the walk possible. Plain DNS answers NXDOMAIN on demand with nothing about other names; DNSSEC cannot sign on demand without an online key, so denial is precomputed as one NSEC per gap between adjacent names, and those records chain every name to the next. NSEC3 hashing stops the online walk, but it costs the operator extra records and signing work, and it is not secrecy: an enumerator can still count the records to learn the zone size and take the hashes offline for a dictionary attack. The real fix for secret names is to keep them out of public DNS.

## Phase 6: Last-Mile Response Forgery

### Design choices

The resolver validates, but the hop from resolver to client carries no proof of it: a stub trusts whatever appears to come from its resolver. So the attacker moves onto the client subnet (`172.16.1.100`) and a route on `edge-router` (not ARP spoofing) sends a client's inbound replies through it. Upstream is the Phase 3 signed, validating resolver, untouched. Two victims: `client-a`, a plain stub using `curl` and the OS resolver, and `client-b`, which validates itself with `delv` against its own root anchor.

### Attack

```
# Client A (plain stub):
client-a example.com      -> 172.16.1.30   (rogue "HACKED" page)
client-a www.example.com  -> 172.16.1.20   (untargeted, untouched)

# Client B (validating), same forgery on the wire:
plain dig on client-b -> 172.16.1.30                        # would be fooled
delv on client-b      -> RRSIG failed to verify
                         resolution failed: RRSIG failed to verify
```

### Report answers

DNSSEC validates upstream but does not stop this, because validation happens at the resolver while trust is placed by the stub. The resolver validated the real answer and set the `ad` flag, but `ad` is one bit in a cleartext packet on the last hop with nothing signing it, so an on-path attacker rewrites the answer and sets `ad` for free,

and the stub believes it. Client B does not rely on `ad`: it runs its own validation against a trust anchor it holds, so it needs real signatures, which the attacker cannot produce, and the forged A is rejected. A plain `dig` on the same machine is fooled, so the difference is the local validation, not the network. Protection comes from validating where trust is placed, or from authenticating the hop itself, which is Phase 7.

# Phase 7: Encrypting the Last Mile (DoT / DoH)

## Design choices

We stand up a private CA (`pki/gen-ca.sh`): an RSA-2048 root CA issues the resolver a certificate whose CN and SAN are `resolver.cs6008.local`, with a DNS SAN only and no IP SAN. The CA goes into the client's system trust store with `update-ca-certificates`, so there is no `-k` or insecure bypass. The client reaches the resolver by name, never by IP (`extra_hosts: resolver.cs6008.local -> 172.16.2.10`), so the certificate is checked against the hostname. On the resolver we add `tls local-tls { cert-file; key-file; }` with `listen-on port 853 tls` for DoT and `listen-on port 443 tls ... http` for DoH. Plain port 53 stays up so the downgrade attempt has something to fall back to. Phase 3 DNSSEC validation still runs upstream.

## Verify

```
# DoT:  TLS session (TLS1.3)...  status: NOERROR, ad  example.com A 172.16.1.20  @853(TLS)
# DoH:  HTTP session (HTTP/2-POST .../dns-query status: 200)  @443(HTTPS)
# enforced: wrong hostname -> handshake failed (certificate error); missing CA -> fails
# no cleartext on edge-router during a DoT lookup: :53 = 0 packets, :853 = 31
```

## Attack: three attempts against two client policies

`strict` uses DoT only and never falls back; `lax` prefers DoT but falls back to cleartext :53 when the encrypted transport cannot be reached (a bad cert is an attack, not an unavailable transport, so it does not trigger fallback). Measured:

| attempt | strict (refuses cleartext) | lax (permits cleartext) |
|---|---|---|
| (i) alter answer in transit | no (TLS integrity) | no (TLS integrity) |
| (ii) rogue-cert impersonation | no (cert not trusted) | no (cert not trusted) |
| (iii) block transport, downgrade | no, fails closed (DoS) | YES, forged to rogue |

(i) The DNS is inside TLS, so the cleartext forge tool sees nothing and a capture of 853 has zero readable `example.com` strings; both clients get the real answer. (ii) A rogue server presents a cert for the right name but self-signed, not from the lab CA; both clients reject it (`handshake failed, certificate error`). (iii) Blocking 853/443 and forging :53 downgrades only the lax client, which falls back to cleartext and takes `172.16.1.30`; the strict client fails closed.

## Report answers

Five of six combinations fail. The one that lands, (iii) against lax, does not beat the cryptography; it beats a client policy that still allows cleartext. We deployed both DoT and DoH and lead with DoT: DoT on its own port 853 is simple to audit and just as simple to spot and block (which is what staged attempt iii), while DoH on 443 blends into HTTPS and resists blocking at the cost of being harder to audit and heavier. An encrypted, authenticated last mile protects the client-resolver hop only. It does nothing for the resolver-to-authoritative path (still plain :53, DNSSEC gives integrity not confidentiality there), it does not hide which resolver you use, and it does not stop a malicious or misconfigured resolver from lying over a valid TLS session: the channel is authenticated, the answer is not. Only client-side DNSSEC validation catches that.

## Appendix: Generative AI usage

AI assistance was used It wrote the Compose files, BIND configs and zones, the signing scripts„ the zone walker, the Phase 7 PKI and DoT/DoH setup.

Typical prompts were plain descriptions of each phase, for example , “write a C/libpcap on-path forge”, “add a strip mode and a no-DS signer option”, and “set up DoT/DoH and the three last-mile attacks”.

Nothing was taken on trust. Every phase was deployed on Docker and checked against the brief’s own pass/fail criteria before being written up, and all the commands are in each phase’s `scripts/`.